

Review



- Last Class:
 - Performance measurement (CPU time, CPI, Amdahl's Law)
- Today's class:
 - Review of instruction set architecture (ISA)
 - Instruction format
 - Addressing mode

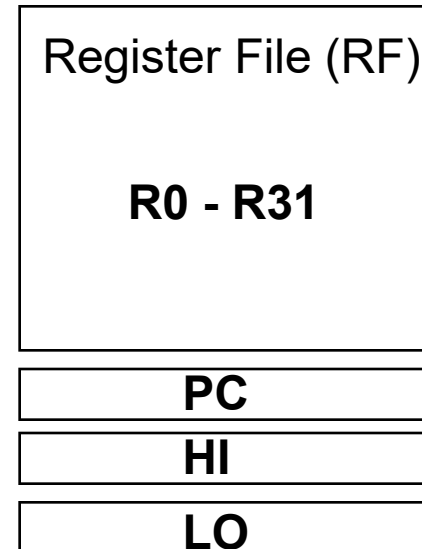
RISC vs CISC

- RISC = Reduced Instruction Set Computer
 - MIPS, SPARC, PowerPC, ARM (Cortex), etc.
- CISC = Complex Instruction Set Computer
 - X86 is the only surviving example
- Goals in the 1980s – reduce design time, faster/smaller implementation, ISA processor/compiler co-design
- ISAs are measured by how well compilers use them, not by how well or how easily assembly language programmers use them
- There are (or, at least, it's believed there are) many old and useful programs that only exist as machine code, so supporting old ISAs has economic value

- **Instruction Categories**

- Computational
- Load/Store
- Jump and Branch
- Floating Point
 - » coprocessor
- Memory Management
- Special

Registers



3 Instruction Formats: all 32 bits wide

op	rs	rt	rd	sa	funct	R format
op	rs	rt	immediate			I format
op	jump target					J format

Aside: MIPS Register Convention

Name	Register Number	Usage	Preserve on call?
\$zero	0	constant 0 (hardware)	n.a.
\$at	1	reserved for assembler	n.a.
\$v0 - \$v1	2-3	returned values	no
\$a0 - \$a3	4-7	arguments	yes
\$t0 - \$t7	8-15	temporaries	no
\$s0 - \$s7	16-23	saved values	yes
\$t8 - \$t9	24-25	temporaries	no
\$gp	28	global pointer	yes
\$sp	29	stack pointer	yes
\$fp	30	frame pointer	yes
\$ra	31	return addr (hardware)	yes
\$cr0-cr8	9	X86 reserved	n.a. (4)

MIPS Instruction Fields

- MIPS fields are given names to make them easier to refer to

op	rs	rt	rd	shamt	funct
----	----	----	----	-------	-------

op	6-bits	opcode that specifies the operation
rs	5-bits	register file address of the first source operand
rt	5-bits	register file address of the second source operand
rd	5-bits	register file address of the result's destination
shamt	5-bits	shift amount (for shift instructions)
funct	6-bits	function code augmenting the opcode

Example

- **MIPS instruction**

- Add \$8, \$9, \$10

0	9	10	8	0	32
---	---	----	---	---	----

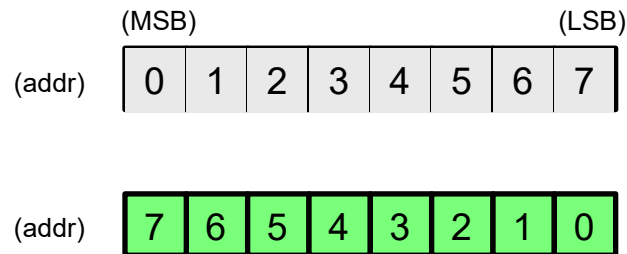
000000	01001	01010	01000	00000	100000
--------	-------	-------	-------	-------	--------

- Hex representation: **012A 4020**
- Decimal representation: **19,546,144**

Memory addressing



- Most modern machines are *byte-addressable* -- yet most memory and cache traffic is in terms of words.
- A natural alignment problem (the start of a word or a double word).
 - Compiler is responsible
 - hardware does the checking
- How are bytes addressed within a word?
 - Big Endian -- address byte 0 is the MSB (IBM, MIPS, SPARC)
 - Little Endian -- address byte 0 is the LSB (vax, intel 80x86)



Addressing Modes



1) Register

operand = content of register = (R)

Add R1, R2

2) Immediate

operand = in instruction = C

Add R1, #54

3) Register indirect

operand = in memory = Mem[(R)]

address = content of register

Add R1, (R2)

4) Displacement (or base)

operand in memory = Mem[(R) + Base]

address = content of register + base

Add R1, 54(R2)

5) Indexed

operand in memory = Mem[(R) + (IR)]

address = content of R + content of IR

Add R1, (R2+R3)

6) Direct (absolute)

operand in memory = $\text{Mem}[C]$

address = a constant in the instruction

Add R1, (1000)

7) Memory indirect

operand in memory = $\text{Mem}[\text{Mem}[(R)]]$

address = the content of $\text{Mem}[(R)]$

Add R1, @(R2)

8) Auto-increment (or decrement)

operand in memory = $\text{Mem}[(R)]$

The content of R is incremented

Add R1, (R2)+

9) Scaled

operand in memory = $\text{Mem}[C+(R)+(IR)*d]$

Add R1, 100(R2)(R3)

Which addressing mode is most suitable for:

stack operations

Scaled

array operations

Auto-increment

Pointers

Register indirect

branch addresses

Register

branch condition evaluation

Memory indirect

address translation

Immediate

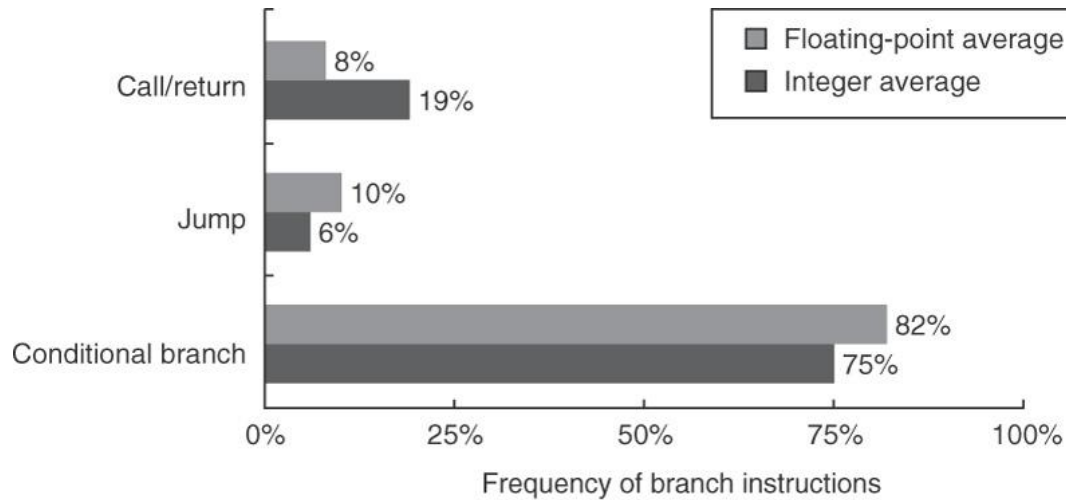
Operations

- **Arithmetic/logical:** add, sub, mult, div, shift (arith,logical), and, or, not, xor ...
 - **Data movement:** copy, move, load, store, ..
 - **Control:** branch, jump, call, return, trap, ...
 - **System:** OS and memory management (ignore for now)
 - Floating point:
 - **Decimal:** legacy from COBOL
 - **String:** move, copy, compare, search
 - **Graphics:** pixel operations, compression, ...
-
- What is the most used instruction(s) in applications?

Frequent Operations

Rank	80x86 instruction	Integer average (% total executed)
1	load	22%
2	conditional branch	20%
3	compare	16%
4	store	12%
5	add	8%
6	and	6%
7	sub	5%
8	move register-register	4%
9	call	1%
10	return	1%
Total		96%

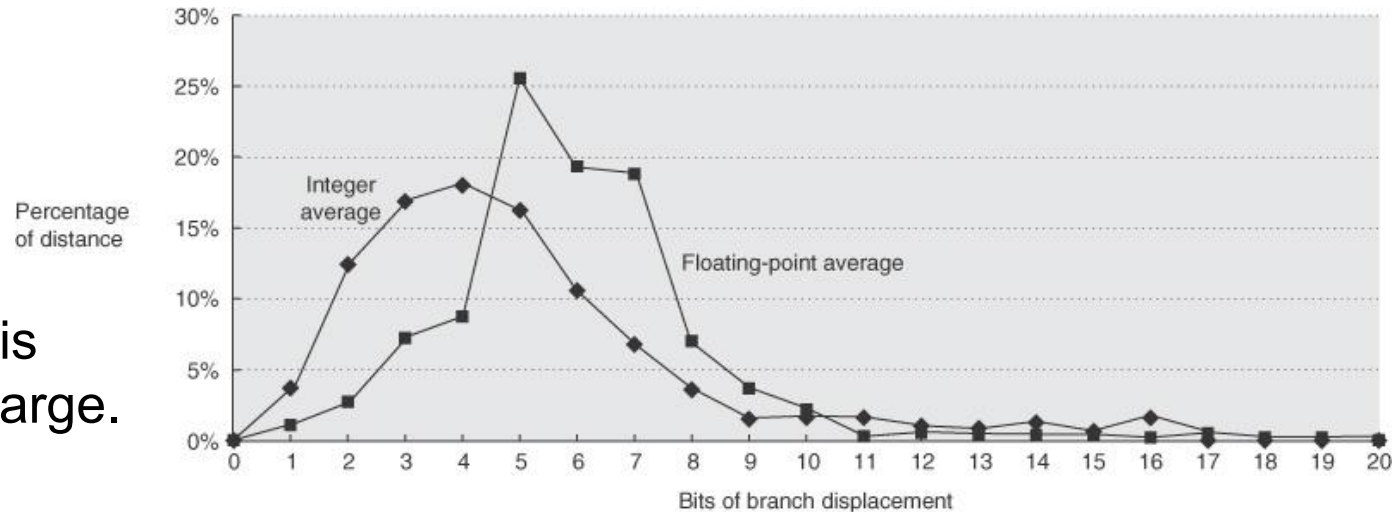
Branch instructions



© 2007 Elsevier, Inc. All rights reserved.

- Branch conditions are usually simple equality/inequality comparisons
- More than 80% of comparisons use immediate constants,
- A majority of comparisons is with “zero”

- Branch distance is usually not very large.



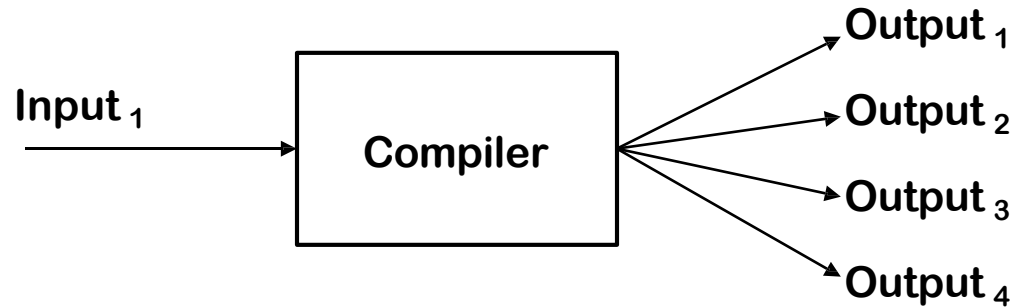
© 2007 Elsevier, Inc. All rights reserved.

Role of compilers



- Compilers are multi-phase: Front-end, high-level optimization, global optimization and code generation.
- The goals of a compiler are: correctness, speed of compiled code, speed of compilation...
- Compiler can do better optimization when instructions are simple
- Allocation of variables:
 - registers are used for temporaries, and possibly parameter passing
 - stacks are used for activation records and local variables
 - a global data area (may be bottom of stack) is used for globals
 - a heap is used for dynamically declared data

What does optimization do?



- The compiler can produce many outputs for a given input
 - The user might want the fastest code
 - The user might want the smallest code
 - The user might want the code that pages least
 - The user might want the least power consuming code
 - The user might want the code that ...
- Optimization tries to reshape the code to better fit the user's goal

A "Typical" RISC



- Fixed format instruction
- General purpose registers -- some have overlapping register windows.
- 3-address, reg-reg arithmetic instruction
- Single address mode for load/store:
base + displacement (no indirection)
- Simple branch conditions -- use PC relative mode for branching.
- Pipelined execution (one instruction issue every clock tick).

RISC II (Berkeley) had 39 instructions, 2 addressing modes and 3 data types, Vax had 304 instructions, 16 addressing modes, 14 data types,

RISC II programs were 30% larger than Vax programs but 5 times as fast.
The RISC compiler were 9 times faster than the Vax compiler.

The RISC-V architecture

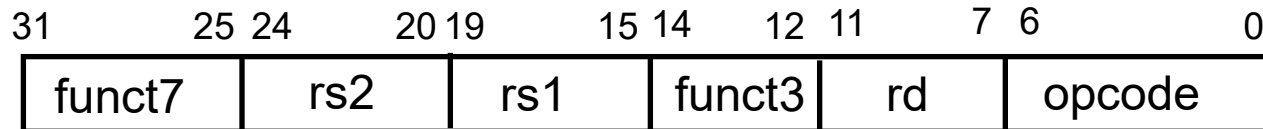


- 32, 64-bit general purpose registers (GPRs)
 - called $x_0, \dots, x_{31}$ (x_0 is hardwired to the value 0).
- 32, 64-bit floating point registers - FPRs (each can hold a 32-bit single precision or a 64-bit double precision value)
 - called $f_0, f_1, \dots, f_{31}$ (or $f_0, f_2, \dots, f_{30}$).
- A few special purpose registers (example: floating point status),
- Byte addressable memories with 64-bit (or 48-bit) addresses.
- 32-bit instructions
- Only immediate and displacement addressing modes (12-bit field)

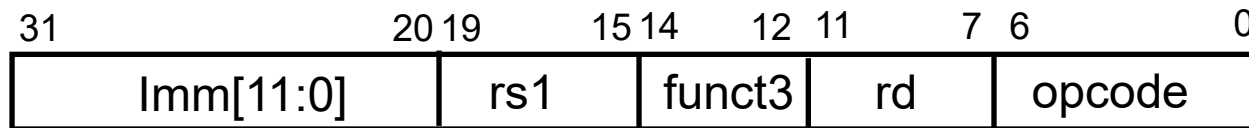
- **Data transfer operations:** `ld, lw, lb, lh, flw, sd, sw, sb, sh, fsw, ...`
- **Arithmetic/logical operations:** `add, addi, sub, subi, slt, and, andi, xor, mul, div, ...`
- **Control operations:** `beq, bne, blt, jal, jalr, ...`
- **Floating point operations:** `fadd, fsub, fmult, fsqrt, ...`

RISC-V instruction format

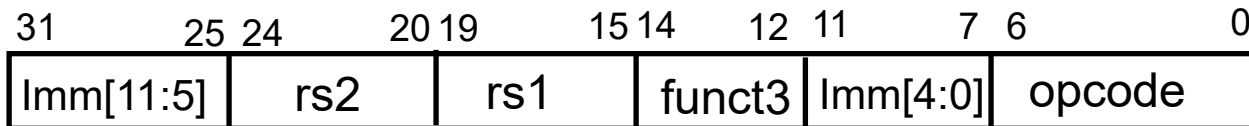
Register-Register (R-type) – used mainly for ALU instructions



Register-Immediate (I-type) – used mainly for load instructions

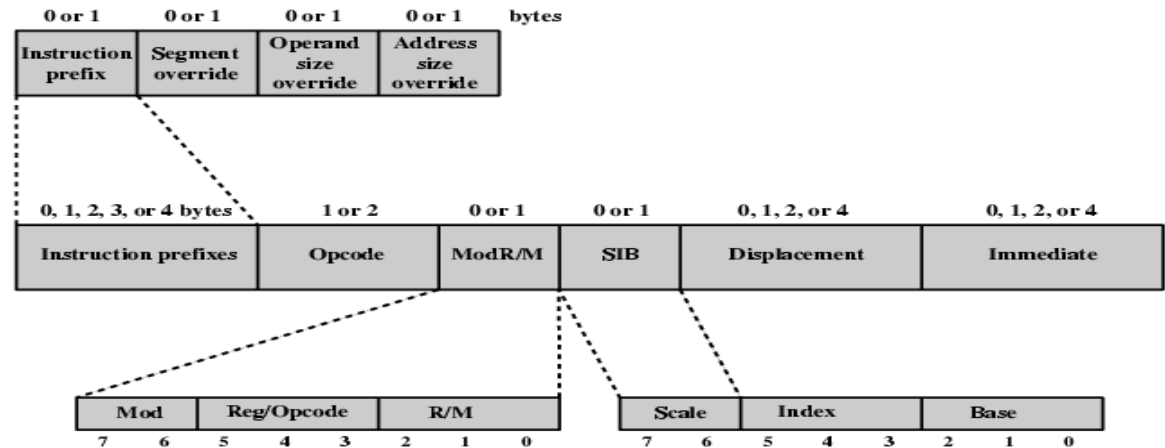


Register-Immediate (S-type) – used mainly for Store



The Intel original 80x86 architecture

- Variable length op-code and instructions



- addressing modes:
 - absolute
 - register indirect (BX, SI, DI in 16-bit modes, extended registers in 32-bit mode)
 - base mode (BX, SI, DI, SI + displacement which is 8, 16 bits , or 8, 16, 32 bits)
 - indexed BX+SI, BX+DI, BP+SI, BP+DI
 - based indexed (indexed+ 8 or 16 bit displacement)
 - based plus scaled indexed (on 386, scale = 0,1,2,3 , restrictions on register use is removed)
 - based with scaled index and displacement.